DP-OPS-0010: Systems Architecture for Persistent Mastery and Context-Isolated Skill Subsystems

I. Theoretical Framework and Strategic Objectives

1.1 The Imperative of Persistent Mastery in Stateless Architectures

The fundamental challenge in managing autonomous agent interactions within a complex repository architecture is the tension between "statelessness" and "mastery." In the Stela repository, the governance model explicitly defines the system as stateless: "Context is not memory; it is a file".[1] This creates a paradox. High-functioning development requires the accumulation of knowledge—preferences, architectural patterns, verification rituals, and security invariants—yet the system resets its state with every session. Without a mechanism for persistence, the "Integrator" (the lead AI agent) suffers from amnesia, forcing the "Operator" (the human) to repeatedly re-litigate established decisions or manually enforce patterns that should be implicit.[1]

This phenomenon, termed "Context Rot," leads to operational inefficiency and, more critically, "Platform Drift." Drift occurs when the agent, lacking a crystallized definition of "correctness," invents new patterns that diverge from the established canon. To combat this, we introduce the concept of **Persistent Mastery**: the crystallization of learned workflows into version-controlled artifacts that serve as an external memory bank.[1] However, unlike standard documentation which explains *what* the system is, these artifacts (Skills) define *how* the system behaves under specific production conditions.

The establishment of the Skills Subsystem via Dispatch Packet OPS-0010 (DP-OPS-0010) addresses this by creating a dedicated repository tone lane (docs/library/skills/) for these artifacts. This transition moves the platform from reliance on implicit, machine-local state (which is forbidden by the "Non-Goals" of the project scope) to explicit, file-based governance.[1]

1.2 The Theory of Context Isolation and Hazard Mitigation

While establishing a library of mastery is essential, the mechanism of its ingestion presents a significant architectural risk. In the Stela "Manifest Supremacy" model, the ops/lib/manifests/CONTEXT.md file dictates the "Single Source of Truth" (SSOT) that is auto-loaded into the agent's context window at the start of every session.[1] The immediate impulse might be to wire the new Skills Library directly into this manifest to ensure the agent is always "smart."

However, DP-OPS-0010 explicitly prohibits this based on the principle of **Context Isolation**. The rationale is that skills are domain-specific. A skill defining "Next.js App Router Structure" (S-LEARN-02) or "React Immutability Patterns" (S-LEARN-04) is highly relevant when working on a production payload in projects/, but it constitutes a **Context Hazard** if loaded when the agent is performing platform maintenance in ops/.[1]

If a "Production Payload" skill is globally loaded, the agent may hallucinate that a Bash script in ops/bin/ requires a React-style component structure, or it might attempt to enforce TypeScript linting rules on a Python governance script. This cross-contamination of concerns degrades the agent's ability to reason clearly about the immediate task. Therefore, the Skills Subsystem is architected as a "Hazardous Material" vault: accessible, highly potent, but strictly **on-demand**. It must never be auto-loaded. This constraint is encoded in the "CONTEXT HAZARD" warning required in the SKILL.md dashboard.[1]

1.3 Scope and Governance Boundaries

The scope of DP-OPS-0010 is strictly delineated to prevent "Platform Drift." The mandate is to establish the subsystem for **Production Payloads** (specifically Next.js/Supabase architectures) only. It explicitly excludes Stela Platform maintenance workflows.

This distinction is crucial for maintaining the "Tone Lanes" defined in docs/GOVERNANCE.md.[1] The "Ops Lane" is characterized as "checklist-heavy, precise, dry, no-fluff," while the "Project Lane" (where skills apply) involves creative problem solving within strict constraints (immutability, TDD, security). By isolating the skills, we reinforce the boundary between the *Operating System* (Stela) and the *User Space* (Projects).

The implementation adheres to the **Stela Filing Doctrine** (The 4-Line Heuristic) [1]:

1. **Does it RUN the system?** -> ops/lib/skill/skill_lib.sh (The extraction logic).
2. **Does it EXPLAIN the system?** -> docs/library/skills/ (The crystallized knowledge).
3. **Is it the WORK itself?** -> The skills target projects/ payloads.
4. **Is it TEMPORARY trash?** -> No, these are permanent artifacts.

II. The Governance Model and Artifact Crystallization

2.1 The Stela Constitution and Filing Doctrine

The Stela repository operates under a strict constitution defined in TRUTH.md.[1] This constitution establishes invariants that must be respected during the creation of any new subsystem. The most relevant invariant for DP-OPS-0010 is: **"Reuse-first; run a duplication check before creating new artifacts."**

The Skills Subsystem is not a duplication of existing documentation; it is a refinement. Where

docs/ provides rationale and expanded guidance, docs/library/skills/ provides **executable logic** in narrative form. These files are designed to be ingested by an LLM to "patch" its behavioral logic. For example, S-LEARN-01 (Verification) is not just a description of testing; it is an imperative algorithm ("Stop if fail") that binds the agent to a specific behavioral loop.[1]

2.2 The "Context Hazard" Protocol

The introduction of the "Context Hazard" protocol represents a maturation of the Stela context management strategy. Previously, context was binary: either inside CONTEXT.md (and thus present) or outside (and thus non-existent). DP-OPS-0010 introduces a tertiary state: **Registered but Latent**.

The SKILL.md dashboard serves as the physical manifestation of this state. It is placed at the repo root for high visibility to the human Operator, but it carries the CONTEXT HAZARD warning header to signal to the AI Integrator that ingestion is forbidden unless explicitly authorized for a specific DP.[1] This "Human-in-the-Loop" gating mechanism ensures that the heavy cognitive load of the skills library is only incurred when necessary, preserving token windows for the actual work.

2.3 Manual vs. Automatic Crystallization

A key decision in this architecture is the reliance on **Manual Crystallization** via skill_lib.sh rather than automatic hooks. While external research suggests "Continuous Learning" systems can automatically extract patterns using "Stop hooks" and "SessionComplete" triggers [1], Stela's governance model prioritizes **Explainability over Automation**.

Automatic extraction risks polluting the repository with "hallucinated" skills or low-value patterns (e.g., "one-time fixes").[1] By forcing the Operator to manually invoke ops/lib/skill/skill_lib.sh, we introduce a friction that acts as a quality filter. Only patterns that have proven their value across multiple sessions and are verified as "Canon-compliant" will be logged to SKILL.md. This aligns with the "Canon Supremacy" principle: changes to the repository structure or logic must be deliberate and traceable.

III. Architectural Implementation

3.1 Directory Structure and Ontology

The implementation establishes a new directory structure within the docs/library/ tree. This placement is intentional. docs/library/ is defined as the "Curated Library" of operator-facing guidance.[1] Placing skills here, rather than in ops/, reinforces that they are informational references for the agent, not executable binaries for the machine.

The Canonical Tree Implementation:

repo_root/

├── SKILL.md # Root-level Dashboard & Hazard Warning

├── docs/

│ └── library/

│ ├── INDEX.md # Registry Pointer

│ └── skills/ # The Skills Registry

│ ├── S-LEARN-01.md # Verification Loop

│ ├── S-LEARN-02.md # Project Guidelines

│ ├── S-LEARN-03.md # Zenith Spec (Reference)

│ ├── S-LEARN-04.md # Coding Standards

│ └── S-LEARN-05.md # Security Protocols

└── ops/

└── lib/

└── skill/

└── skill_lib.sh # Manual Pattern Extraction Tool

3.2 The Root Dashboard (SKILL.md)

SKILL.md acts as the "Switchboard" for the subsystem. Its placement at the root ensures that an Operator executing ls -F immediately sees the available mastery tools. However, its exclusion from ops/lib/manifests/CONTEXT.md ensures it remains optically visible but cognitively isolated.

Design constraints for SKILL.md:

- **Header:** Must contain the "CONTEXT HAZARD" warning.[1]
- **Scope:** Strictly for Production Payloads (projects/).
- **Function:** Logs candidate patterns for future crystallization.

3.3 The Registry Update (docs/library/INDEX.md)

The docs/library/INDEX.md file serves as the "Authorized Surface Registry".[1] To comply with the "Library Guard" (tools/lint_library.sh), all new artifacts in docs/library/skills/ must be explicitly registered here. This prevents "Ghost Artifacts"—files that exist on disk but are not recognized by governance.

The update to INDEX.md will create a new section: --- 5. Skills (Production Payloads / On-Demand) ---. This section will list pointers to S-LEARN-01 through 05 and SKILL.md, formally inducting them into the repository canon without mandating their global loading.

IV. The Skills Library: Detailed Artifact Analysis

The core of DP-OPS-0010 is the content of the skills themselves. These files represent the "Crystallized Mastery" of the organization. Each skill is designed to address a specific vector of "Drift."

4.1 S-LEARN-01: The Verification Loop Skill

Quality assurance in AI-assisted development often fails due to the "drift" of small, uncorrected errors accumulating over time. S-LEARN-01 codifies a strict **Six-Phase Verification Loop** that transforms verification from an ad-hoc activity into a mandatory ritual.[1]

The Six Phases of Verification:

Phase	Activity	Command / Tool	Success Criteria
1. Build	Compilation Check	npm run build	Zero errors. Stop immediately on failure.
2. Type	Static Analysis	npx tsc --noEmit / pyright.	Zero critical type errors.
3. Lint	Stylistic Analysis	npm run lint / ruff check.	Adherence to config.
4. Test	Functional Integrity	npm run test -- --coverage	**80% Minimum Coverage**.
5. Security	Secret/Log	grep for sk-,	Zero exposed

	Scanning	api_key, console.log	secrets or debug logs.
6. Diff	Contextual Review	git diff --stat	No unintended file touches.

Rationale: The requirement for "Stop immediately on failure" in Phase 1 is critical. It prevents the AI from attempting to "fix" code by adding more logic on top of a broken build, which inevitably leads to hallucinated solutions. The 80% coverage threshold in Phase 4 is a hard gate for "Production Payloads," ensuring that velocity does not compromise stability.[1]

4.2 S-LEARN-02: Project Guidelines (The Physics of Production)

S-LEARN-02 defines the "Physics" of the production environment. By rigidly defining the technology stack and file structure, we prevent the AI from introducing incompatible technologies (e.g., suggesting a MongoDB adapter in a Supabase project).[1]

Stack Definition:

- **Frontend:** Next.js 15 (App Router), TypeScript, React, TailwindCSS.
- **Backend:** FastAPI (Python 3.11), Pydantic.
- **Database:** Supabase (PostgreSQL).
- **Cache:** Redis.
- **Deployment:** Google Cloud Run, Vercel.

Architectural Constraints: The skill enforces specific directory patterns: frontend/src/app for routes and backend/routers for API logic. Crucially, it imposes a "Physical" constraint on file size: **"Many small files" (200-400 lines)**.[1] This constraint is optimized for AI context windows; smaller files are easier for an agent to read, reason about, and refactor without losing context.

4.3 S-LEARN-03: The Zenith Specification (Reference Implementation)

While S-LEARN-02 provides the rules, S-LEARN-03 provides the example. It uses the **Zenith** platform (an AI-powered customer discovery tool) as the "Gold Standard" for implementation.[1] This allows the AI to "mimic" a proven architecture rather than inferring one from first principles.

Key Reference Patterns:

- **API Protocols:** Usage of a generic ApiResponse class to standardize JSON envelopes (Success, Data, Error).
- **AI Integration:** A specific pattern for using the Claude API with "Tool Calling" to force structured JSON output validated by Pydantic models (AnalysisResult).

- **State Management:** The use of a custom useApi hook in React to manage the loading/data/error states of async operations, preventing race conditions.

Strategic Value: By pointing the AI to S-LEARN-03, the Operator can say "Implement this feature using the Zenith pattern," drastically reducing the token cost of explaining the desired architecture.

4.4 S-LEARN-04: Coding Standards (Style and Semantics)

This skill enforces the "Style" of the code, moving beyond syntax (handled by linters) to semantics (readability and logic).[1]

Core Standards:

- **Immutability:** The skill marks this as **Critical**. It mandates the use of the spread operator (...) over mutation methods (.push, .splice). This is vital for React state consistency.
- **Naming Conventions:** Enforces "Verb-Noun" patterns (fetchMarketData) to ensure code is self-documenting ("Readability First").
- **React Patterns:** Explicitly bans "Ternary Hell" (nested conditionals) in favor of clear short-circuiting (&&) or early returns.
- **Code Smells:** Defines hard limits for function length (>50 lines) and nesting depth (>5 levels) to force modularity.

4.5 S-LEARN-05: Security Protocols (The 10 Domains)

Security in autonomous coding cannot be an afterthought. S-LEARN-05 provides a 10-point mandatory checklist that covers the entire attack surface of a modern web application.[1]

The 10 Security Domains:

Domain	Requirement	Implementation
1. Secrets	No hardcoding.	Use .env. Verify existence at runtime.
2. Input	Validate ALL inputs.	Use zod schemas. Validate file size/MIME.
3. SQLi	No string concatenation.	Use Parameterized queries or ORM (.eq()).
4. Auth	Secure Token Storage.	httpOnly Cookies (No localStorage).

5. XSS	Sanitize HTML.	DOMPurify + CSP Headers.
6. CSRF	Prevent Cross-Site Forgery.	CSRF Tokens + SameSite=Strict.
7. Rate Limit	Prevent Abuse.	Throttle all APIs. Aggressive limits on search.
8. Data Exposure	Protect PII in logs.	Redact secrets/PII. Generic user errors.
9. Blockchain	Solana-specific safety.	Verify signatures, recipient, and balance.
10. Dependencies	Supply Chain Security.	npm audit, commit package-lock.json.

This skill integrates external best practices from the **OWASP Top 10** [2] and PortSwigger's **Web Security Academy** [1], tailoring them to the specific stack (Next.js/Supabase) defined in S-LEARN-02.

V. Operational Tooling: The Extraction Logic

5.1 The skill_lib.sh Utility

To operationalize the "Manual Crystallization" strategy, DP-OPS-0010 introduces ops/lib/skill/skill_lib.sh. This Bash script serves as the bridge between the transient "Work Log" in TASK.md and the permanent "Candidate Log" in SKILL.md.

Script Logic:

The script is a simple, stateless utility that accepts three arguments: Pattern Name, Context, and Solution. It appends a formatted Markdown entry to the SKILL.md dashboard.

```
Bash
```

```
#!/usr/bin/env bash
# ops/lib/skill/skill_lib.sh
# Purpose: Manually log pattern candidates to SKILL.md.

SKILL_DASHBOARD="SKILL.md"

#... (Validation Logic)...

# Append to Candidate Log
cat <<EOF >> "$SKILL_DASHBOARD"

### $NAME
* **Context:** $CONTEXT
* **Problem:** (Auto-logged)
* **Solution:** $SOLUTION
* **Date:** $(date +%Y-%m-%d)
EOF
```

5.2 Operational Workflow

The Operator uses this tool during the "Closeout" phase of a Dispatch Packet. If a new pattern was discovered (e.g., a specific way to handle Supabase real-time subscriptions in React 19), the Operator runs:

./ops/lib/skill/skill_lib.sh "Supabase Realtime Hook" "React 19 Hydration" "Use distinct useEffect cleanup..."

This creates a candidate entry. In a future "Gardening" DP, these candidates are reviewed, refined, and moved into a formal S-LEARN file. This two-stage process (Log -> Crystallize) prevents the library from becoming a dumping ground for unverified snippets.

VI. The Dispatch Packet (DP-OPS-0010)

The following section contains the **exact text** of the Dispatch Packet to be placed in TASK.md. It is constructed to pass all Stela Repo Gates and strictly adhere to the formatting rules defined in the repository dump.[1]

DP-OPS-0010: Establish Skills Subsystem and Persistent Mastery

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

- **Base Branch:** main
- **Required Work Branch:** work/dp-ops-0010-skills
- **Base HEAD:** (Current HEAD of main)

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

- **Loaded:** OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- **Job-specific canon:**
  - docs/library/INDEX.md (Registry)
  - ops/lib/manifests/CONTEXT.md (Manifest)

II) SCOPE & SAFETY

- **Objective:** Establish the Skills subsystem with strict context isolation to prevent Platform Drift.
- **Non-Goals:**
  - DO NOT wire the skills library into ops/lib/manifests/CONTEXT.md (No auto-loading).
  - DO NOT modify projects/ payloads directly.
- **Target Files allowlist (hard gate):**
  - SKILL.md (new)
  - docs/library/skills/S-LEARN-01.md (new)
  - docs/library/skills/S-LEARN-02.md (new)
  - docs/library/skills/S-LEARN-03.md (new)
  - docs/library/skills/S-LEARN-04.md (new)
  - docs/library/skills/S-LEARN-05.md (new)
  - ops/lib/skill/skill_lib.sh (new)
  - docs/library/INDEX.md
  - SoP.md

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The platform currently relies on implicit AI memory for specialized tasks (verification, security, tech stack), leading to drift and inconsistency.
- **Drift:** Reusable patterns are lost in closed threads.
- **Desired State:** A "Crystallized" library of skills exists in docs/library/skills/ but is strictly isolated from the global context to prevent hazards.

B) REQUEST

1. Create Root Dashboard: SKILL.md

Create SKILL.md at the repo root.

- **Header:** Must include "CONTEXT HAZARD" warning.
- **Content:**
  - Warning: Skills are for Production Payloads only. Invoke manually.
  - Template Section: For logging new candidates.
  - Candidate Log: Empty table for future entries.

2. Populate Skills Library: docs/library/skills/

Create the directory and the following files:

S-LEARN-01 (Verification):

- Define the 6 phases: Build, Type, Lint, Test (80% coverage), Security, Diff.
- Define Output Format: "Verification Report".

S-LEARN-02 (Project Guidelines):

- Define Tech Stack: Next.js 15 (App), Supabase (PG), FastAPI (Python), Redis.
- Define File Structure: frontend/src/app, backend/routers.
- Constraint: "Many small files" (200-400 lines).

S-LEARN-03 (Zenith Spec):

- Reference implementation details from Zenith.
- Patterns: ApiResponse, fetchApi, useApi hook.
- AI: Claude Tool Calling structure.

S-LEARN-04 (Coding Standards):

- Principles: Readability, KISS, DRY, YAGNI.
- Standards: Immutability (spread operator), Verb-noun naming.
- React: Functional updates, useMemo.
- Anti-patterns: Functions > 50 lines, "Ternary Hell".

S-LEARN-05 (Security):

- 10 Domains: Secrets, Input (Zod), SQLi (Parameterized), Auth (httpOnly), XSS (DOMPurify), CSRF, Rate Limit, Data Exposure, Blockchain (Signatures), Dependencies.

3. Create Operational Tool: ops/lib/skill/skill_lib.sh

Create a bash script to log patterns.

- **Function:** Append a structured entry to SKILL.md.

- **Inputs:** Pattern Name, Context, Solution.
- **Constraints:** No auto-hooks. Manual execution only.

4. Update Registry: docs/library/INDEX.md

- Add a new section: --- 5. Skills (Production Payloads / On-Demand) ---.
- Register S-LEARN-01 through 05 and SKILL.md.
- *Note:* Do NOT add to ops/lib/manifests/CONTEXT.md.

5. Update History: SoP.md

- Log the establishment of the Skills subsystem.
- Note the "Context Hazard" governance decision.

C) CHANGELOG

- Added SKILL.md (Root Dashboard).
- Created docs/library/skills/ with S-LEARN-01 through 05.
- Created ops/lib/skill/skill_lib.sh.
- Updated docs/library/INDEX.md (Registry).
- Updated SoP.md (Governance).

D) PATCH / DIFF

(Worker to generate)

E) RECEIPT (REQUIRED)

Verification

- ./ops/bin/dump --scope=platform
- bash tools/context_lint.sh (Must pass; ensures we didn't break existing manifest).
- bash tools/lint_library.sh (Must pass; ensures new skills are registered in INDEX.md).
- bash tools/lint_truth.sh (Must pass).

REQUIRED OUTPUT

- storage/handoff/DP-OPS-0010-RESULTS.md
- git diff --stat

3) CLOSEOUT (MANDATORY)

- [ ] Produce DP-OPS-0010-RESULTS.md.
- [ ] Update SoP.md.
- [ ] Append Work Log entry.

VII. Verification and Compliance Strategy

7.1 Compliance with Stela Repo Gates

The execution of DP-OPS-0010 is subject to the rigorous "Repo Gates" defined in .github/workflows/repo_gates.yml.[1] The verification strategy addresses each gate:

- **Verify Tree (tools/verify_tree.sh):** This script checks for required directories (ops, docs, tools). The addition of docs/library/skills/ complies with the schema. The SKILL.md file at the root is a slight deviation but is permitted as a "Root Dashboard" similar to TRUTH.md.
- **Lint Truth (tools/lint_truth.sh):** This script checks for spelling and forbidden terms in canon files. The skills content must avoid forbidden terms (like the misspelled "Steela").
- **Library Guard (tools/lint_library.sh):** This is the most critical gate for this DP. It strictly enforces that every file in docs/library/ is registered in docs/library/INDEX.md. Failure to update INDEX.md will cause this gate to fail, blocking the PR. The DP explicitly includes this registration step.

7.2 Context Isolation Verification (tools/context_lint.sh)

The tools/context_lint.sh script validates the ops/lib/manifests/CONTEXT.md file.[1]

- **Success Criterion:** The script must pass, validating that all files listed in CONTEXT.md exist.
- **Negative Verification:** The Operator must manually verify that the skills/ directory is **NOT** present in CONTEXT.md. The context_lint script only checks for *missing* files, not *extra* files, so this manual check is vital to enforce the "Context Hazard" protocol.

7.3 Governance Logging (SoP.md)

The SoP.md file is the historical ledger of the project. A specific entry is drafted to document this architectural shift:

2026-02-01 — DP-OPS-0010: Skills Subsystem & Context Isolation

- Purpose: Establish a crystallized skills library for Persistent Mastery without polluting global context.
- What shipped:
  - Created SKILL.md (Root Dashboard) with "Context Hazard" warning.
  - Established docs/library/skills/ containing S-LEARN-01 through 05.
  - Added ops/lib/skill/skill_lib.sh for manual pattern logging.
  - Updated docs/library/INDEX.md to register the new tone lane.
- Verification:
  - tools/lint_library.sh (PASSED)

- tools/context_lint.sh (PASSED - Skills NOT in global context)

VIII. Conclusion

DP-OPS-0010 represents a pivotal evolution in the Stela platform's architecture. It resolves the inherent conflict between the stateless nature of the autonomous agent ("Context is a file") and the need for accumulating domain-specific expertise ("Persistent Mastery").

By establishing the **Skills Subsystem** as a distinct, version-controlled library, we grant the agent access to "Crystallized" knowledge: verified rituals (S-LEARN-01), rigid structural constraints (S-LEARN-02), proven architectural patterns (S-LEARN-03), semantic styling (S-LEARN-04), and non-negotiable security protocols (S-LEARN-05).

Critically, the implementation of the **Context Isolation** protocol and the **Context Hazard** warning ensures that this new capability does not come at the cost of operational stability. The agent remains focused and "light" during platform maintenance, only accessing the "heavy" skills when explicitly directed to work on a production payload. This "separation of concerns" is the hallmark of a mature, scalable Governance-as-Code system, ensuring that Stela remains robust against both "Context Rot" and "Platform Drift."

The "Manual Crystallization" workflow via skill_lib.sh further reinforces the supremacy of the human Operator, ensuring that the Skills Library remains a curated garden of high-value patterns rather than a dumping ground for transient AI hallucinations. This architecture lays the foundation for a system that gets smarter over time without becoming slower or more confusing—the ultimate goal of Persistent Mastery.

Works cited

1. sample_only.sh
2. Countdown to 2027 – maximising use of NAMs in food safety assessment: closing the gap for regulatory assessments in Europe - ResearchGate, accessed January 31, 2026, https://www.researchgate.net/publication/392201802_Countdown_to_2027_-_maximising_use_of_NAMs_in_food_safety_assessment_closing_the_gap_for_regulatory_assessments_in_Europe